Fondamentaux d'Ansible et Idempotence

1. Contexte et architecture Agentless

La configuration manuelle des serveurs via des connexions interactives SSH présente des limites en environnement de production : absence de traçabilité, dérive de configuration (hétérogénéité entre les nœuds) et risque d'erreur humaine.

L'approche de l'Infrastructure as Code (IaC) résout ces problématiques en décrivant l'état attendu des systèmes dans des fichiers textes versionnables. Dans cet écosystème, Ansible se distingue par son architecture sans agent (Agentless).

1.1 Le mécanisme d'exécution

Aucun démon spécifique n'est installé sur les machines cibles (nœuds managés). Le nœud de contrôle (Master) centralise l'intelligence et communique avec les cibles selon un cycle précis :

1. Ouverture d'une connexion SSH standard.
2. Génération d'un micro-script Python autonome intégrant le module sollicité et ses paramètres (mécanisme interne Ansiballz).
3. Transfert temporaire de ce script sur la cible (via SFTP).
4. Exécution locale par l'interpréteur Python de la cible.
5. Retour du résultat structuré (JSON) vers le Master, suivi de l'autodestruction du script temporaire.

Les seuls prérequis sur les nœuds managés sont un service OpenSSH actif et un interpréteur Python 3.

2. Configuration et Inventaire

Un projet Ansible s'appuie sur deux fichiers fondamentaux pour définir son environnement de travail.

2.1 L'inventaire (Inventory)

Le fichier d'inventaire cartographie l'infrastructure. Il regroupe les machines par rôles logiques. Le format INI est le standard historique.

```ini
[web_nodes]
node1 ansible_host=192.168.4.101
```

```
node2 ansible_host=192.168.4.102

[db_nodes]
node3 ansible_host=192.168.4.103

[production:children]
web_nodes
db_nodes

[web_nodes:vars]
ansible_user=formation
ansible_python_interpreter=/usr/bin/python3
```

2.2 Le fichier de configuration (ansible.cfg)

Le comportement du moteur est régi par le fichier `ansible.cfg`. Placé à la racine du projet, il garantit la portabilité de la configuration.

```ini
[defaults]
inventory = ./inventory.ini
remote_user = formation
host_key_checking = False
stdout_callback = yaml

[privilege_escalation]
become = False
become_method = sudo
become_user = root

[ssh_connection]
pipelining = True
```

La directive `pipelining = True` accélère l'exécution en injectant les scripts Python directement via l'entrée standard (stdin) de la session SSH, évitant la phase de transfert SFTP sur le disque distant.

3. Le concept d'idempotence

La fiabilité de l'Infrastructure as Code repose sur la propriété mathématique d'idempotence : $f(f(x)) = f(x)$.

Une opération est idempotente si son exécution répétée aboutit au même état final que sa première exécution, sans provoquer d'effets de bord ni d'erreurs.

Ansible adopte une approche déclarative. L'administrateur décrit l'état final attendu (le paquet doit être présent). Le module interroge le système cible :

- Si le paquet est absent, il l'installe (statut `changed`).
- Si le paquet est déjà présent, il ne fait rien (statut `ok`).

3.1 Utilisation encadrée des modules d'exécution brute

Les modules `ansible.builtin.command` et `ansible.builtin.shell` exécutent directement des instructions système. Ils ignorent l'état préalable de la machine cible et renvoient systématiquement le statut `changed` dès que le code de retour shell vaut 0, brisant la promesse d'idempotence.

L'utilisation de modules natifs (`apt`, `systemd`, `file`) est impérative. En cas de nécessité absolue d'utiliser `command`, l'instruction doit être encadrée par la directive `creates` (pour éviter l'exécution si le fichier cible existe).

4. Les commandes Ad-Hoc

Les commandes Ad-Hoc exécutent une tâche unitaire sur un groupe de machines en ligne de commande. Elles permettent de valider la connectivité, d'auditer l'état du parc ou d'appliquer des correctifs d'urgence.

La syntaxe générale est la suivante : `ansible <groupe_cible> -m <nom_du_module> -a "<arguments>" [options]`

Exemples de gestion courante :

```shell
# Test de connectivité SSH et de l'interpréteur Python
ansible all -m ping

# Lecture d'informations système (module command par défaut)
ansible web_nodes -a "uptime"

# Installation d'un paquet avec élévation de privilèges (-b)
ansible web_nodes -b -m ansible.builtin.apt -a "name=curl state=present"
```

```
# Gestion d'un service
ansible web_nodes -b -m ansible.builtin.systemd -a "name=nginx state=restarted"
```

5. Structure fondamentale d'un Playbook

Pour des configurations persistantes et complexes, les tâches sont regroupées au sein de Playbooks, rédigés en langage YAML. Un Playbook est une liste de scénarios (Plays), associant une liste de tâches à un groupe de machines cibles.

5.1 Syntaxe YAML de base

```yaml
---
- name: Titre du scénario global
  hosts: web_nodes
  become: true

  tasks:
    - name: Installation du serveur web
      ansible.builtin.apt:
        name: apache2
        state: present
        update_cache: true

    - name: Activation et démarrage du service
      ansible.builtin.systemd:
        name: apache2
        state: started
        enabled: true
```

5.2 Exécution et modes d'audit

La commande `ansible-playbook` lit et applique le fichier YAML sur l'inventaire. Elle dispose d'options d'audit sécurisées pour les environnements de production.

Shell

```
# Exécution standard
ansible-playbook deploy_web.yml
```

```
# Vérification syntaxique du fichier YAML sans connexion aux cibles
ansible-playbook deploy_web.yml --syntax-check

# Mode simulation (dry-run) : évalue les changements sans les appliquer
ansible-playbook deploy_web.yml --check
```